

Claude Code 101 - Start Guide til hvordan man bruger Claude Code

indholdsfortegnelse

- Generelt info
- Tab shift
- Explore - code - plan - commit
- En god prompt
- Context usage
- PR
- CLAUDE.md
- Subagenter
- Skills
- MCP
- Hooks
- Potentiel risiko
- Konklusion
- Kommandoliste
- Kilder

Generelt info

Claude Code er en software udviklet af Anthropic (Obs: Claude og Claude Code er IKKE det samme). Den bruges til mange ting som fx at lave apps og hjemmesider, læse og skrive kode, teste projekter og meget mere. I min mening er det en af de bedste værktøjer og have hvis du gerne vil lave hjemmesider, apps og meget mere. Claude Code kan bruges i terminalen (fx Windows Powershell hvis du bruger Windows systemet), VS Code og desktop. Jeg anbefaler at bruge det i terminalen. For at have adgang til Claude Code skal man have abonnement til Claude. Den billigste plan (Pro) koster 18 euro om måneden og er et fantastisk sted at starte. Men hvis du skal lave mange store projekter vil jeg anbefale at købe Max.

Dette er en simpel guide for at hjælpe med at komme i gang uden at gøre det alt for kompliceret. Jeg vil anbefale at man undersøger alt grundigt når man er kommet godt fra start. (Brug evt. *anthropic academy*)

Tab Shift

Brug kommandoen tab shift til at skifte mellem to modes: *accept edits*, Claude spørger ikke længere om tilladelse hver gang den skal have adgang til dine filer, den gør det bare automatisk nu. *Plan mode*, brug ALTID inden du sætter Claude i gang med et stort projekt så du kan gennemgå koden og sørge for at dig og Claude er enige om projektet og om hvad der skal gøres.

Explore - code - plan - commit

Mange mennesker sætter bare Claude direkte i gang med en meget stor kommando uden at gennemgå noget først. Det kan føre til en masse småting som skal fixes senere hen hvilket kan være en meget lang proces. Altid brug reglen *Explore - code - plan - commit*

```

Plan: Simple Blå Hjemmeside

Context
Brugeren ønsker en minimal hjemmeside med ét enkelt interaktionspunkt: tekst der byder velkommen, og en knap der farver hele skærmen blå.

Implementering
En enkelt fil: blaa.html

Indhold:
- Siden starter med hvid baggrund, centreret layout
- Stor, fed tekst: "Klar til at se blå"
- En knap under teksten
- Klik på knappen → hele siden bliver blå (baggrund + tekst forsvinder eller skifter farve)

Teknisk løsning:
- Ren HTML + CSS + vanilla JavaScript (ingen frameworks)
- onclick handler sætter document.body.style.backgroundColor = "blue"
- Tekst og knap skjules/forsvinder når blå-tilstand aktiveres, så kun blå forbliver

Fil der oprettes
- C:\Users\sl\blaa.html

Verifikation
- Åbn blaa.html i en browser
- Bekræft at tekst og knap vises på hvid baggrund
- Klik knappen → hele skærmen skal være ren blå

Claude has written up a plan and is ready to execute. Would you like to proceed?

> 1. Yes, auto-accept edits
2. Yes, manually approve edits
3. No, refine with Ultraplan on Claude Code on the web
4. Tell Claude what to change
   shift+tab to approve with this feedback

```

Start i plan mode (læs: *tab shift*) Skriv din prompt til Claude (Hvis du er i tvivl om hvordan man skriver en god prompt læs: *En god Prompt*) Claude skriver nu hele planen for hvad du vil samt hvordan hjemmesiden vil se ud og en masse andet. I bunden kan du se 4. valgmuligheder. 1. giver Claude frit spil til at gå i gang med hjemmesiden. 2. sætter Claude i gang, men du skal give tilladelse til ændringer i filer osv. 3. Tager dig ind på Claude Code web version hvor du kan finjustere koden. Og 4. giver dig lov til at skrive hvad Claude evt. skal ændre i planen. Det kan være alt fra filer til indhold. Sidste trin, altså at comitte er enlig bare at fortælle Claude at den skal skubbe koden og foretage ændringerne, før du gør dette er det meget vigtigt du også selv har testet at alt er som det skal. Brug evt. en test agent til at hjælpe dig.

Husk ALTID at bruge reglen *Explore - code - plan - commit* før du går i gang med en ændring eller et projekt.

En god Prompt

Når du skriver en prompt til Claude, skal du huske at være meget grundig. Ellers kan det være at Claude misforstår og laver noget andet end det du har i tankerne. Husk og altid at bruge *plan mode* før nogle store ændringer. Eksempel på en god prompt: *I need to add WebP conversion to our image upload pipeline. Figure out where in the pipeline it should happen, whether we need new dependencies, and how to approach it.*

Context Usage

Context er mængden af plads Claude kan have i sin hukommelse. Hver gang Claude laver kode, tjekker filer osv. stiger mængden af context. Der er ikke uendelig context så det er vigtigt hvordan du bruger det.

Når context bliver fyldt op, kompakter Claude automatisk samtalen, hvilket kan føre til tab af små detaljer og andre småting.

For at holde styr på mængden af context brugt kan du bruge forskellige kommandoer til at holde det nede (Læs: *kommandoer*).

måder man kan holde mængden af context nede på er fx. at bruge specifikke og detaljerede prompts.

PR

En PR (Pull Request) er en måde at foreslå ændringer til en kodebase på platforme som GitHub eller GitLab. Når du er færdig med at arbejde på en ny funktion eller en fejlrettelse på en separat branch, åbner du en PR for at bede om, at ændringerne bliver gennemgået og flettet (merged) ind i hovedgrenen. For at lave en PR skal du simpelthen bare spørge Claude om at gøre det. VIGTIGT: Før du skubber en PR skal du altid bede Claude tjekke koden med en subagent (Læs: *Subagenter*)

CLAUDE.md

Claude.MD er en af de vigtigste features i Claude code. I bund og grund er Claude.MD en fil hvor der står projektets struktur, stack og regler Claude skal følge. Hvornår skal man så bruge en Claude.MD fil? Du har faktisk ikke brug for filen til småting som enkeltstående spørgsmål eller små korte projekter. Filen er mere beregnet til store projekter som fx. websites eller apps, hvor det er vigtigt at Claude kan huske forskellige regler, kode og opføre sig ens over mange sessioner.

(Note: Filen hedder faktisk CLAUDE.md og ikke Claude.MD, beklager!)

Subagenter

Forestil dig at Claude Code er en projektleder som skal bygge et hus. Selvfølgelig bygger lederen ikke bare huset alene, lederen har selvfølgelig folk til at arbejde for sig som kan hjælpe med at bygge huset. Fx. En VVS'er, en Murer og en Tømrer. Det samme gælder, når Claude koder. Den skriver nemlig ikke hele koden alene. Den bruger noget der hedder Subagenter. Hvad gør subagenter? Subagenter kan bruges til mange ting, at tjekke koden igennem, planlægge projekter, skubbe opdateringer osv. Hvordan opretter jeg og bruger subagenter? Det er meget simpelt. Claude Code kan faktisk gøre begge dele. Du kan bede den om at oprette forskellige subagenter. Fx. hvis du gerne vil have et friskt par "øjne" til at tjekke koden igennem, så kan du bede Claude om at oprette en subagent der kan tjekke kode igennem. Og du behøver faktisk slet ikke at tænke på hvornår du skal bruge dem, for det gør Claude nemlig helt af sig selv.

Skills

Skills er specialiserede kommandoer der fortæller Claude præcis hvordan en bestemt opgave skal løses fx laver `/review` automatisk en grundig kodegennemgang uden at du skal forklare hvad Claude skal kigge efter.

MCP

MCP giver AI-modeller som Claude en struktureret måde at forbinde sig til eksterne værktøjer, databaser og tjenester. Uden MCP er Claude begrænset til det, der er i kontekstvinduet. Med MCP kan Claude læse og skrive filer, databaser, eller kode, kalde eksterne API'er (fx. GitHub), hente realtidsdata osv.

Fordelen er at MCP-servere er genbrugelige og deles på tværs af klienter. Den samme server virker i Claude Code, Claude Desktop, og andre MCP-kompatible klienter.

Du laver en MCP via kommandoen: `claude mcp add`

Hooks

Hooks er automatisk udførte shell-kommandoer der kører på bestemte tidspunkter i Claude Codes livscyklus. De giver dig deterministisk kontrol, i modsætning til instruktioner i CLAUDE.md som Claude kan vælge at følge, kører hooks altid. Hvad bruger man så hooks

til? Hooks kan bruges til en del ting fx. sende notifikationer når Claude er færdig med noget, blokere farlige kommandoer, logge ændringer til audit-logs osv. Hvis du gerne vil sætte hooks op vil jeg anbefale at snakke med Claude Code om det, den kan finde de rette hooks og sætte dem op for dig.

Potentiel Risiko

Selvom alt dette er virkelig smart og fedt, er det dog vigtigt for mig at sige at det potentielt kan være en smule "farligt" at give en AI agent lov til at ændre i dine filer. Men selvfølgelig er det meget sjældent at der sker noget, så jeg vil overhovedet ikke være bekymret.

Konklusion

Dette er alle de ting du skal vide for at komme godt fra start med Claude Code. Dog er der nogle ting i guiden som jeg har simplificeret for at gøre det lettere at forstå. Jeg vil derfor stærkt anbefale at besøge <https://www.anthropic.com/learn> for at lære mere eller få en dybere forståelse af nogle af emnerne.

Kommandoliste

Dette er en liste over nogle basale kommandoer, der er tonsvis af kommandoer du kan bruge i Claude Code, men denne liste indeholder nogle simple og vigtige kommandoer.

`/compact` - kompakter samtalen så Claude ikke bruger for meget context storage

`/clear` - sletter alt fra sessionen (godt hvis du skal til at arbejde på en ny feature)

`/context` - viser hvor meget af Claudes context der er brugt op

`/init` - Opretter en CLAUDE.md fil til dit projekt

`/review` - Gennemgår en pull request

`/security-review` Sikkerhedsgennemgang af kodeændringer

`/simplify` - Forbedrer kodekvalitet og reducerer kompleksitet

`/schedule` - Planlægger opgaver der kører automatisk

`/update-config` - Ændrer Claude Code indstillinger

`/ultrareview` - Dybdegående multi-agent kodegennemgang

`claude mcp add` - Laver ny MCP server

/mcp - giver dig overblik over dine MCP servere

/hooks - giver dig overblik over dine hooks

Kilder

<https://www.anthropic.com/learn>

<https://claude.ai/new>

<https://www.anthropic.com/>

<https://chatgpt.com/>

<https://substack.com/home/post/p-181523848>

Held og lykke med Claude Code!

Researchet og skrevet af Storm:)